

คำอธิบายเครื่องหมายดังแสดงในตารางต่อไปนี้

ลำดับ	สัญลักษณ์	ความหมาย
1	+	เป็นการกำหนด access mode เป็น public
2	-	เป็นการกำหนด access mode เป็น private
3	#	เป็นการกำหนด access mode เป็น protected
4	Class	บ่งบอกว่าเป็นคลาสปกติ
5	Abstract Class*	ชื่อคลาสเป็นคำเดียวเท่านั้น บ่งบอกว่าเป็นคลาสไม่สมบูรณ์
6	<<Interface>>	บ่งบอกว่าเป็นอินเตอร์เฟซ
7	←	บ่งบอกการสืบทอด extends
8	←..	บ่งบอกการ implements
9	◊	บ่งบอกการเป็นส่วนหนึ่ง composite
10	methodname() *	คำเดียว บ่งบอกว่าเป็นเมธอดไม่สมบูรณ์
11	attribute หรือ methodname()	ตัวหนา บ่งบอกความเป็น final
12	attribute หรือ methodname()	ตัวขีดเส้นใต้ บ่งบอกความเป็น static

byte → short → Int → long → float → double
char ↗ Int

class

extends

class

interface

implements

class

interface

extends

interface

Writing Code

*.java file

Java Source Code

Compile

Compiling Code

*.class file

Byte Code

Interpret

Running Code

machine code

Output

- “Compile-Time Polymorphism” เรียกอีกอย่างว่า “Static Polymorphism” ซึ่งการเรียกใช้เมธอดจะได้รับการกำหนดช่วงเวลาคอมไพล์ การ Compile-Time Polymorphism สามารถทำได้โดยอาศัยหลักการ Overloading เมธอด
- “Runtime Polymorphism” เรียกอีกอย่างว่า “Dynamic Binding” หรือ “Dynamic Method Dispatch” ในกระบวนการการเรียกใช้เมธอดจะได้รับการกำหนดหรือปรับเปลี่ยนแบบ Dynamic ขณะรันไทม์ การ “Runtime Polymorphism” สามารถทำได้โดยอาศัยหลักการ Overriding เมธอด

Component	Interface	Abstract Class
แอททริบิวต์	Constructor	✗
	Static	✓
	Non-Static	✗
	Final	✓
	Non-final	✗
เมธอด	Abstract	✓
	Static	✓
	Non-Static	✓
	Final	✗
	Non-final	✓
Access Modifier	private	✗
	default	✓
	protected	✗
	public	✓

ชนิดข้อมูล	ขนาด (Bit/Byte)	ช่วงค่า	ค่าเริ่มต้น
boolean	1 / Not Positively defined	true หรือ false	false
char	16 / 2	Unicode character 'u0000' ถึง 'uFFFF'	'u0000'
byte	8 / 1	-128 ถึง +127	0
short	16 / 2	-32,768 ถึง +32,767	0
int	32 / 4	-2 ³¹ ถึง +2 ³¹ -1	0
long	64 / 8	2 ⁶³ ถึง 2 ⁶³ -1	0L
float	32 / 4	IEEE 754 floating point standard -3.40E+38 ถึง +3.40E+38	0.0f
double	64 / 8	IEEE 754 floating point standard -1.79E+308 ถึง +1.79E+308	0.0d หรือ 0.0

modifier	คลาสเดียวกัน	คลาสที่อยู่ใน package เดียวกัน	คลาสที่เป็น subclass	คลาสใด ๆ
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

ประเภทสีเดียวกัน

ตัวแปรชนิดข้อมูลพื้นฐาน

byte

short

int*

long

float

double*

char

boolean

จำนวนเต็ม

จำนวนทศนิยม

ตัวอักษร

ค่าความจริง

ตัวแปรชนิดข้อมูลแบบอ้างอิง

String (ข้อความ)

Object (วัตถุ)

Array (อาร์เรย์)

SIMPLE TYPE	WRAPPER CLASSES
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long

	ประเภท	บ่งบอกถึง	ความหมาย
this	คีย์เวิร์ด	คลาสตนเอง	ใช้เพื่อเรียกใช้งานแอททริบิวต์หรือเมธอดภายในคลาส ซึ่งนิยมเรียกใช้งานเมื่อเกิดความคลุมเครือเท่านั้น และควรใช้กับแอททริบิวต์
super	คีย์เวิร์ด	คลาสแม่	ใช้เพื่อเรียกใช้งานแอททริบิวต์และเมธอดของคลาสแม่
this(...)	เมธอด	คลาสตนเอง	เมธอดที่เรียกใช้งานเพื่อเรียกคอนสตรัคเตอร์ภายในคลาส
super(...)	เมธอด	คลาสแม่	เมธอดที่เรียกใช้งานเพื่อเรียกคอนสตรัคเตอร์ของคลาสแม่

เป็นการอนุญาตให้คลาสลูก (Subclass) สามารถเพิ่มเติมการทำงานเฉพาะ (เพิ่มโค้ด) ในเมธอดที่คลาสแม่ (Superclass) มีไว้แล้วได้ โดยมีเงื่อนไขดังนี้

ข้อที่ 1

จำนวนและชนิดของ argument จะต้องเหมือนกัน

ข้อที่ 2

ชนิดข้อมูลของค่าที่ส่งกลับ จะต้องเหมือนกัน

ข้อที่ 3

Access Modifier จะต้องไม่ต่ำกว่าเดิม

Parent

Child

Upcasting

Vs

Downcasting

```
Parent a = new Child();
```

Upcasting คือการแปลงชนิดข้อมูล (typecasting) รูปแบบหนึ่งที่พยายามแปลงเอา 객체จากคลาสลูก (Child Object) ไปเป็นออบเจกต์ของคลาสแม่ (Parent Class Object) ซึ่งการทำ Upcasting นิยมนำมาใช้เมื่อต้องการเข้าถึงแอททริบิวต์หรือเมธอดของคลาสลูก อย่างไรก็ตาม นักศึกษาสามารถเข้าถึงแอททริบิวต์หรือเมธอดทั้งหมดได้ มีสิทธิ์เข้าถึงเพียงแอททริบิวต์หรือเมธอดของคลาสลูกเท่านั้น

Overloading

- เปลี่ยนจำนวน Parameter ที่รับเข้ามา
- เปลี่ยน DataType ของ Parameter

DataType - Casting (x = ตัวแปล)

- > : String.valueOf(x);
- String > Double : Double.parseDouble(x)
- String > Int : Integer.parseInt(x);
- String > Float : Float.parseFloat(x);
- String > char : "x".charAt(0)

Encapsulation

- การใช้ Access Modifier
- การใช้ getter , setter

Inheritance

- (Class) ทนสืบทอดโดยใช้ Extends
- (Interface) สืบทอดโดยใช้ Implemets

Polymorphism

- ทน Overloading / Overriding

การเรียกใช้เมธอดภายในคลาสเดียวกัน



